

Implantação, VM e atualizações pelo GitHub

Controle de Brindes

Este guia explica como colocar o sistema em produção em uma máquina virtual Linux, configurar o banco e o e-mail, operar backups e atualizar o código com commits controlados no GitHub.

Versão do documento: 1.0

Data: 26/09/2026

Ambiente recomendado: VM Linux + PHP-FPM + MySQL/MariaDB

1. Resumo executivo

O Controle de Brindes é uma aplicação PHP com banco relacional, interface responsiva para computador e tablet, fluxo de solicitações, estoque, aprovações, protocolos e integrações com Lecom e e-mail.

Para produção, a arquitetura recomendada é uma VM Linux com PHP-FPM, Nginx, MySQL ou MariaDB, armazenamento persistente e backups fora do servidor. O Vercel continua adequado para demonstração e homologação, mas não deve ser tratado como banco de produção sem Blob ou outro armazenamento persistente.

Decisão recomendada

Produção: VM da empresa + MySQL/MariaDB + storage persistente. GitHub: repositório privado com revisão e histórico de commits. Vercel: demo/homologação, com BLOB_READ_WRITE_TOKEN configurado se os dados precisarem persistir.

2. Arquitetura alvo

Camada	Responsabilidade	Onde fica
Navegador	Bootstrap, Alpine.js, formulários, loading e outbox offline.	Computador e tablet
Web	Nginx recebe HTTPS e encaminha PHP para PHP-FPM.	VM Linux
Aplicação	Rotas, autenticação, serviços, Lecom, fila de e-mails e relatórios.	Código PHP em /var/www/controle-brindes
Dados	Solicitações, estoque, usuários, auditoria e configurações.	MySQL/MariaDB
Arquivos	Fotos, notas fiscais, assinaturas, PDFs, logs e backups.	storage/ em disco persistente
Código	Commits, branches, tags e revisão de mudanças.	GitHub privado

3. Checklist de pré-requisitos

A implantação deve ser feita com contas da empresa, não com contas pessoais do desenvolvedor.

- VM Linux Ubuntu 22.04/24.04 ou distribuição equivalente, com 2 vCPU, 4 GB de RAM e pelo menos 30 GB de disco SSD.
- Acesso SSH por chave, usuário operacional sem uso diário de root e firewall liberando apenas SSH, HTTP e HTTPS.
- PHP 8.2 ou superior com pdo_mysql, mbstring, gd, fileinfo, openssl, zip, xml, dom e intl recomendado.
- Nginx ou Apache; este manual usa Nginx + PHP-FPM.
- MySQL 8.0.16+ ou MariaDB 10.4+, usando InnoDB e utf8mb4.
- Composer, Git, unzip, cron e Certbot para certificado HTTPS.
- Domínio corporativo, por exemplo brindes.empresa.com.br, apontado para o IP da VM.
- Caixa de e-mail corporativa para o remetente do sistema e autorização da TI para SMTP AUTH ou OAuth/Graph.

4. Contas, segredos e variáveis

Os valores abaixo são exemplos. Não coloque senhas, API keys, tokens, certificados ou o arquivo .env no GitHub.

Variável	Obrigatória	Observação
APP_URL	Sim	URL pública com HTTPS.
APP_KEY	Sim	64 caracteres hexadecimais; gerar uma vez e guardar com a TI.
APP_TIMEZONE	Sim	America/Sao_Paulo.
DB_HOST/PORT/NAME/USER/PASS	Sim	Acesso ao MySQL/MariaDB de produção.
MAIL_*	Para e-mail	SMTP ou integração Microsoft 365; ver seção 8.
LECOM_API_KEY	Se fallback	Somente no servidor; nunca no navegador.
LECOM_TEST_MODE	Homologação	true apenas no ambiente de homologação.
BLOB_READ_WRITE_TOKEN	Somente Vercel	Persistência da SQLite demo; não é necessário na VM.

Gere a chave da aplicação no próprio servidor:

```
php -r "echo bin2hex(random_bytes(32)), PHP_EOL;"
```

Modelo mínimo de .env para produção em VM:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://brindes.empresa.com.br
APP_KEY=COLE_A_CHAVE_GERADA_AQUI
APP_TIMEZONE=America/Sao_Paulo

DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=brindes
DB_USER=brindes_app
DB_PASS=SEGREDO_FORTE

MAIL_DRIVER=smtp
MAIL_HOST=smtp.office365.com
MAIL_PORT=587
MAIL_ENCRYPTION=tls
MAIL_USERNAME=caixa-do-sistema@empresa.com.br
MAIL_PASSWORD=SEGREDO_FORA_DO_GIT
MAIL_FROM_ADDRESS=caixa-do-sistema@empresa.com.br
MAIL_FROM_NAME="Controle de Brindes"
```

Regra de segurança

O arquivo .env deve ficar apenas na VM e em um cofre de segredos da empresa. Se uma chave vazar, revogue-a e gere outra imediatamente.

5. Preparar a VM Linux

Os comandos abaixo são um roteiro para Ubuntu. A TI deve adaptar os nomes de pacotes à distribuição escolhida.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y nginx mariadb-server git unzip composer certbot \
python3-certbot-nginx php php-fpm php-mysql php-mbstring php-gd \
php-curl php-xml php-zip php-intl
```

Criar o usuário de operação e a pasta da aplicação:

```
sudo adduser --system --group --home /var/www/controle-brindes deploy
sudo mkdir -p /var/www/controle-brindes
sudo chown -R deploy:www-data /var/www/controle-brindes
sudo chmod 750 /var/www/controle-brindes
```

Configurar firewall básico:

```
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Recomendações de segurança da VM:

- usar chave SSH e desabilitar login de root por senha;
- manter o sistema operacional e PHP atualizados;
- não expor MySQL na internet; permitir conexão somente local ou pela rede privada;
- usar fail2ban ou solução equivalente para tentativas repetidas de SSH;
- manter o relógio do servidor sincronizado via systemd-timesyncd ou chrony.

6. Baixar o código e instalar dependências

O repositório deve ser privado e pertencer à empresa. A VM pode usar deploy key somente de leitura para baixar o código, ou uma credencial de máquina com escopo mínimo.

```
sudo -u deploy git clone https://github.com/EMPRESA/gestaobrindes.git \
/var/www/controle-brindes
cd /var/www/controle-brindes
sudo -u deploy composer install --no-dev --optimize-autoloader
sudo -u deploy cp .env.example .env
sudo chmod 600 .env
sudo chown deploy:www-data .env
```

Importante

O document root do Nginx deve ser /var/www/controle-brindes/public. Nunca aponte o domínio para a raiz do projeto, pois app/, database/ e storage/ não devem ficar públicos.

7. Criar o banco e instalar a aplicação

Crie um banco exclusivo e um usuário sem privilégios administrativos globais:

```
sudo mariadb

CREATE DATABASE brindes CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'brindes_app'@'localhost' IDENTIFIED BY 'SEGREDO_FORTE';
GRANT ALL PRIVILEGES ON brindes.* TO 'brindes_app'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Instalação recomendada com usuário administrador corporativo:

```
cd /var/www/controle-brindes
php bin/install.php --admin-email=ti@empresa.com.br --admin-password='SenhaForteTemporaria' --admin-name='Administrador TI'
```

Alternativa manual pelo banco:

```
mysql -u brindes_app -p brindes < database/schema.sql
mysql -u brindes_app -p brindes < database/seed.sql
```

Depois da instalação:

- troque a senha do administrador no primeiro acesso;
- cadastre usuários reais e atribua perfis;
- confirme que APP_KEY, APP_URL e APP_TIMEZONE estão corretos;
- garanta permissão de escrita somente em storage/;
- não execute --fresh nem --demo em produção.

8. Configurar Nginx, PHP-FPM e HTTPS

Crie um server block semelhante ao abaixo e ajuste a versão do socket PHP-FPM instalada:

```
server {
    listen 80;
    server_name brindes.empresa.com.br;
    root /var/www/controle-brindes/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
    }

    location ^~ /storage { deny all; }
}
```

```
sudo ln -s /etc/nginx/sites-available/controle-brindes \
/etc/nginx/sites-enabled/controle-brindes
sudo nginx -t
sudo systemctl reload nginx
sudo certbot --nginx -d brindes.empresa.com.br
```

Validação rápida

Abra /api/health, faça login e teste a lista de brindes. O cadeado HTTPS deve aparecer antes de cadastrar dados reais.

9. E-mail Microsoft 365

O código usa PHPMailer e a fila cron/send_notifications.php. Para o envio real, a TI precisa autorizar a caixa e decidir o método de autenticação.

Item	Valor/requisito
Servidor	smtp.office365.com
Porta	587
Segurança	TLS/STARTTLS
Conta	caixa técnica da empresa
Política	SMTP AUTH habilitado para a caixa, se permitido; ou OAuth/Graph
Remetente	igual à conta ou com permissão Send As

A Microsoft recomenda autenticação moderna/OAuth. Se o tenant bloquear autenticação básica, a configuração username/password do PHPMailer não funcionará; nesse cenário, migrar o Mailer para OAuth2 ou Microsoft Graph.

Configure o cron da fila:

```
* * * * * /usr/bin/php /var/www/controle-brindes/cron/send_notifications.php
# opcional: redirecionar a saída para um log do sistema
```

Faça um teste com um solicitante real e confirme o recebimento, o remetente, o assunto e o registro de falha no log.

Referências Microsoft

Authenticated SMTP: <https://learn.microsoft.com/en-us/exchange/clients-and-mobile-in-exchange-online/authenticated-client-smtp-submission>

SMTP para Microsoft 365: <https://learn.microsoft.com/en-us/exchange/mail-flow-best-practices/how-to-set-up-a-multifunction-device-or-application-to-send-email-using-microsoft-365-or-office-365>

OAuth para SMTP: <https://learn.microsoft.com/en-us/exchange/client-developer/legacy-protocols/how-to-authenticate-an-imap-pop-smtp-application-by-using-oauth>

10. Lecom em homologação e produção

Na homologação, o botão usa o cookie LecomSSOTicket no navegador e o fallback server-side `/api/lecom/process/start`. As variáveis de API devem ficar somente no servidor.

```
LECOM_TEST_MODE=true  
LECOM_API_KEY=SEGREDO_NO_AMBIENTE  
LECOM_TEST_USER=USUARIO_DE_TESTE
```

Antes de produção, validar processo 26, versão 10, ambiente, permissões e o retorno do número da instância. Não registrar API key, ticket ou senha nos logs.

11. Backups e restauração

O backup precisa cobrir banco e arquivos. O SQL sozinho não recupera fotos, notas fiscais, assinaturas e PDFs gravados em storage/.

```
# Banco - exemplo diário  
10 2 * * * /usr/bin/php /var/www/controle-brindes/cron/backup.php  
  
# Cópia externa recomendada: enviar storage/backups e storage/uploads  
# para OneDrive corporativo, S3 ou outro armazenamento separado.
```

Rotina mínima:

- guardar pelo menos uma cópia diária fora da VM;
- testar restauração mensalmente em uma base separada;
- copiar storage/uploads, storage/signatures e storage/pdf junto com o SQL;
- tratar o SQL como dado confidencial por conter nomes, e-mails e registros operacionais;
- antes de qualquer migração, congelar movimentações e registrar o horário do último backup.

Restauração:

```
php bin/restore-backup.php storage/backups/backup-AAAAMDD-HHMMSS.sql
```

Após restaurar, conferir login, brindes, movimentações, protocolo PDF, anexos e envio de e-mail.

12. Atualizações usando GitHub

O GitHub deve ser a fonte do código. A VM apenas baixa uma versão aprovada; não edite arquivos PHP diretamente em produção.

Fluxo recomendado para uma alteração:

- criar uma branch: feature/nome-da-mudanca ou fix/nome-do-problema;
- implementar localmente e executar lint/testes;
- fazer commits pequenos e descritivos;
- abrir Pull Request para revisão;
- após aprovação, integrar em main e criar tag quando for uma versão;
- fazer backup antes de atualizar a VM;
- executar `git pull --ff-only` na VM e atualizar dependências;

- validar /api/health e o checklist funcional.

Exemplo de atualização na VM:

```
cd /var/www/controle-brindes
sudo -u deploy git fetch origin
sudo -u deploy git checkout main
sudo -u deploy git pull --ff-only origin main
sudo -u deploy composer install --no-dev --optimize-autoloader
sudo systemctl reload php8.2-fpm
sudo systemctl reload nginx
```

Commits recomendados:

```
fix: corrige horario das solicitacoes
feat: adiciona notificacao ao solicitante
docs: atualiza manual de implantacao
```

13. Rollback e mudanças de banco

Antes de uma atualização que altera dados ou schema:

- gerar backup SQL e cópia de storage/;
- anotar o commit atual: `git rev-parse HEAD`;
- testar a atualização em homologação;
- aplicar migração em janela combinada;
- se necessário, reverter código com `git revert` ou retornar a uma tag;
- não tentar corrigir banco de produção com `git reset --hard`.

Consultar o commit atual:

```
git log -5 --oneline
git status --short
```

Regra de rollback

Código pode voltar por Git. Dados só voltam por restauração de backup. Por isso uma mudança de banco sempre precisa de backup testado antes.

14. Checklist de go-live

- [] VM em conta da empresa e acesso SSH por chave
- [] Domínio apontado e HTTPS válido
- [] PHP 8.2+ e extensões conferidos
- [] Banco criado com usuário de aplicação
- [] .env preenchido e protegido
- [] APP_KEY guardada com a TI
- [] Admin corporativo criado e senha trocada
- [] SMTP Microsoft 365 validado
- [] Lecom homologação validado

- [] Cron de e-mail, alertas e backup ativo
- [] Backup externo e teste de restauração concluídos
- [] GitHub privado com revisão de mudanças
- [] /api/health, login, solicitação, aprovação, estoque e PDF testados

15. Documentos relacionados no projeto

Documento	Conteúdo
docs/Manual-de-Instalacao-e-Migracao.md	Instalação PHP/MySQL e cron.
docs/Backup-e-Restauracao.md	Backup do banco, storage e restauração.
docs/TRD.md	Arquitetura técnica e integrações.
docs/Esquema-Backend.md	Camadas, dados e contratos backend.
docs/Plano-de-Implementacao.md	Fases e critérios de conclusão.
docs/Contas-e-Credenciais.md	Propriedade de contas e credenciais.

Conclusão

Com VM, banco persistente, SMTP corporativo, backups e GitHub privado, a empresa consegue operar o sistema e atualizar o código sem depender de uma conta pessoal ou de dados armazenados em uma demo efêmera.